

By Alexander Madrid and Riley Bunch

Pennsylvania State University

AI100 — Spring 2026

1. What I Learned About GenAI in This Project

This project helped me understand how Generative AI behaves when working alongside real machine-learning code. One of the biggest lessons was realizing that GenAI doesn't "see" code the way a compiler does — it reasons about patterns, assumptions, and logic. When I introduced bugs into my Fashion-MNIST CNN, GenAI didn't simply point to the broken line. Instead, it guided me by asking questions that forced me to think more deeply.

For example, in my original script I correctly normalized and reshaped the dataset:

```
trainX = (trainX / 255.0).astype("float32")[..., None] testX = (testX / 255.0).astype("float32")[..., None]
```

When I later removed the channel dimension as part of a bug, GenAI didn't just say "your shape is wrong." It asked whether the preprocessing still produced a 4-D tensor, reminding me that CNNs expect input shaped like (batch, height, width, channels). This helped me understand why the model broke, not just where.

Another example came from the loss function. My correct model used:

```
loss="sparse_categorical_crossentropy"
```

When I intentionally changed it to `categorical_crossentropy` during the bug-insertion phase, GenAI reminded me that this loss expects one-hot encoded labels, while Fashion-MNIST provides integer labels. This reinforced how important it is to understand the assumptions behind each component.

I also learned that GenAI is excellent at helping me reflect on my own reasoning. When I wrote initial reflections, it evaluated whether my thinking was shallow or thorough. This made me more aware of when I was guessing versus when I was analyzing the problem.

Finally, I learned that GenAI can sound confident even when it's wrong. This project taught me to treat GenAI as a reasoning partner, not an authority. It's most useful when I use it to:

- Clarify my thinking
- Explore alternative explanations
- Understand interactions between model components
- Reflect on mistakes

But I still have to verify everything against the actual code and results.

2. What I Learned About Building an AI System

Working with the Fashion-MNIST CNN taught me how sensitive AI systems are to small changes. A single line of code can completely change the model's behavior.

For example, the model architecture in my script was:

```
model = Sequential([ Conv2D(32, (3,3), activation="relu", input_shape=(28,28,1)),  
MaxPooling2D((2,2)), Flatten(), Dense(64, activation="relu"), Dense(10, activation="softmax") ])
```

When I changed the final layer during debugging exercises — such as removing the softmax activation — the model produced raw logits instead of probabilities. This caused unstable training and incorrect predictions. It showed me how tightly coupled each layer is to the loss function and evaluation metrics.

I also learned how important the training configuration is. In my working script, I trained with:

```
history = model.fit( trainX, trainy, epochs=2, batch_size=64, validation_split=0.3, verbose=2 )
```

When I later changed `batch_size` to something extreme (like 1000), the model updated its weights only once per epoch. This dramatically reduced learning and produced poor accuracy. It taught me that hyperparameters are not just “settings” — they fundamentally change how the model learns.

Another example came from the prediction logic:

```
prediction = model.predict(testX[:1]) predicted_label = labels[np.argmax(prediction)]
```

When I changed the slice to `testX[5:6]` during the bug-insertion phase, the predicted label no longer matched the image I expected. This helped me understand how easy it is to misinterpret results if the indexing is off by even one element.

Finally, generating the confusion matrix:

```
y_pred = model.predict(testX).argmax(axis=1) cm = confusion_matrix(testy, y_pred)
```

showed me how evaluation tools can reveal deeper issues. When the model was misconfigured, the confusion matrix immediately exposed widespread misclassification.

Overall, this project taught me that building an AI system is a systematic engineering process. It requires:

- Understanding data formats
- Matching model architecture to the task
- Choosing appropriate loss functions
- Setting correct hyperparameters
- Verifying outputs at every stage

It's not just about writing code — it's about understanding how every component interacts with the rest of the system.